



خلاصه‌ی اسلایدهای فصل ۵، جلسه‌ی دوم

در ابتدای این جلسه دوباره روی الگوریتم Dijkstra تمرکز می‌کنیم. ایده‌ی کلی این است که از یک node مبدأ شروع می‌کنیم، در هر مرحله نزدیک‌ترین node کشف‌نشده را به مجموعه‌ی نودهای نهایی‌شده اضافه می‌کنیم و فاصله‌ی بقیه‌ی نودها را با استفاده از این نود جدید به‌روزرسانی می‌کنیم. در مثال‌های اسلایدها، با نگاه کردن به ستون‌های جدول، می‌بینیم که برای هر node یک پیشینی (predecessor) نگه می‌داریم و در نهایت با دنبال کردن همین پیشینی‌ها، درخت کوتاه‌ترین مسیر (shortest-path tree) را می‌سازیم؛ بعد هم از روی همین درخت، جدول forwarding table نود مبدأ ساخته می‌شود، یعنی برای هر مقصد مشخص می‌شود از کدام link باید بسته‌ها را بفرستد. در مثال دوم، مرحله‌به‌مرحله مجموعه‌ی N' (نودهایی که مسیرشان قطعی شده) بزرگ‌تر می‌شود و نشان می‌دهد که چطور وزن‌ها و پیشینی‌ها در هر مرحله به‌روزرسانی می‌شوند.

در اسلاید «Dijkstra's algorithm, discussion» پیچیدگی الگوریتم بررسی می‌شود. اگر تعداد نودها را n بگیریم، در پیاده‌سازی ساده، در هر تکرار باید همه‌ی نودهای خارج از N' را چک کنیم، که در مجموع حدوداً $\frac{n(n+1)}{2}$ مقایسه داریم، یعنی مرتبه‌ی زمانی $O(n^2)$. اگر از ساختارهایی مثل priority queue استفاده کنیم، می‌توانیم این را به $O(n \log n)$ بهتر کنیم. بعد اسلایدها به یک نکته‌ی مهم اشاره می‌کنند: اگر link cost به مقدار ترافیک روی آن لینک وابسته باشد، ممکن است الگوریتم باعث oscillation در مسیرها شود. در شبکه‌ی چهار نودی شکل، وقتی ترافیک بر اساس کوتاه‌ترین مسیر پخش می‌شود، cost لینک‌ها عوض می‌شود و در نتیجه الگوریتم دوباره مسیرهای جدید پیدا می‌کند؛ این تغییر مسیرها دوباره ترافیک را عوض می‌کند و همین‌طور جلو می‌رود و ممکن است به یک رفتار ناپایدار برسیم که مسیرها مدام عوض شوند.

بعد، اسلاید یک نمای کلی از فصل می‌دهد و نشان می‌دهد که بعد از الگوریتم‌های link-state (مثل Dijkstra)، حالا نوبت الگوریتم‌های distance-vector است. الگوریتم Distance Vector Routing یک الگوریتم توزیع‌شده است؛ هر router فقط با همسایه‌های مستقیم خودش حرف می‌زند، نه با کل شبکه. این الگوریتم «تکراری» (iterative) است، یعنی تا وقتی که هیچ نودی اطلاعات جدیدی نفرستد، روند ادامه دارد، و «ناهمگام» (asynchronous) است، یعنی لازم نیست همه‌ی نودها هم‌زمان آپدیت شوند؛ هر نودی هر وقت تغییر حس کرد، کار خودش را می‌کند.

هستهی تئوری این الگوریتم، معادله ی Bellman-Ford است. اگر $d_x(y)$ را هزینه ی کم هزینه ترین مسیر از نود x به نود y در نظر بگیریم، و $c(x, v)$ را هزینه ی link بین x و همسایه اش v ، آن وقت داریم

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\},$$

که در آن کمینه روی همه ی همسایه های x گرفته می شود. یعنی بهترین مسیر x به y این است که از بین همه ی همسایه های x نگاه کنیم ببینیم اگر اول برویم سراغ هر کدام، مجموع «هزینه ی رسیدن از x به آن همسایه» و «هزینه ی بهترین مسیر آن همسایه تا y » چقدر می شود، و کوچک ترین این ها را انتخاب کنیم. نودی که این مینیمم را می دهد، همان next hop در جدول مسیریابی است. در مثال گراف شش نودی (u, x, y, v, w, z) به کمک همین فرمول نشان داده می شود که مثلاً $d_u(z) = 4$ و این که از روی همین محاسبه می توانیم تصمیم بگیریم که برای رسیدن به z از کدام همسایه باید استفاده کنیم.

در الگوریتم distance-vector هر نود x یک distance vector به نام D_x نگه می دارد که شامل تخمین هزینه ی کمینه از x به همه ی نودهای دیگر است. یعنی $D_x(y)$ یک تقریبی از همان $d_x(y)$ واقعی است. نود x هزینه ی لینک خودش به هر همسایه ی مستقیم را می داند، یعنی مقادیر $c(x, v)$ را دارد، و علاوه بر آن کپی هایی از distance vector همسایه ها را هم ذخیره می کند، یعنی D_v برای همه ی همسایه های v . هر از چند وقت، هر نود distance vector خودش را برای همسایه ها می فرستد. وقتی نود x یک DV جدید از همسایه ی v دریافت کند، برای هر مقصد y مقدار $D_x(y)$ خودش را با معادله ی Bellman-Ford به روزرسانی می کند:

$$D_x(y) \leftarrow \min_v \{c(x, v) + D_v(y)\}.$$

اگر بعد از این محاسبه، مقدار جدید با قبلی فرق داشته باشد، یعنی DV نود x عوض شده، و در این صورت x هم نوبت خودش است که این DV جدید را برای همسایه ها بفرستد. تحت شرایط طبیعی (مثلاً این که هزینه ها همیشه غیرمنفی باشند و مرتب عوض نشوند)، این روند بالاخره همگرا می شود و $D_x(y)$ به همان مقدار واقعی $d_x(y)$ نزدیک می شود.

یک نکته ی مهم در پیاده سازی، ساختار distance table هر نود است. برای هر نود، یک جدول داریم که سطرهايش مقصدهای ممکن هستند و ستون هایش همسایه های مستقیم آن نود. هر خانه ی جدول، یعنی $d_E(dest, via)$ ، نشان می دهد اگر نود E بخواهد به آن مقصد برسد و اولین پرش را از طریق آن همسایه برود، هزینه ی کل مسیر چقدر تخمین زده می شود. در مثال نود E ، با داشتن distance table اولیه و هزینه ی لینک ها، نشان داده می شود که چطور بعضی ورودی ها می توانند به مسیرهای دارای حلقه برگردند (مثلاً مسیرهایی که دوباره از خود E عبور می کنند). بعد، از روی همین جدول فاصله، جدول routing یا forwarding ساخته می شود: در هر سطر (یعنی برای هر مقصد) کمترین مقدار انتخاب می شود و همسایه ای که این مینیمم از ستون او آمده است، تبدیل به next hop آن مقصد می شود. این همان جدول نهایی ای است که router برای تصمیم گیری در مورد ارسال بسته ها از آن استفاده می کند.

در ادامه، اسلایدها روی رفتار الگوریتم distance-vector در طول زمان تمرکز می کنند. یک مثال سه نودی با نودهای x, y و z نشان می دهد که چطور distance table در زمان های $t = 0, t = 1, t = 2$ قدم به قدم به روزرسانی می شوند. در ابتدا هر نود فقط هزینه های لینک های مستقیم خودش را می داند و بقیه ی مقصدها را بی نهایت در نظر می گیرد. بعد از تبادل چند پیام DV، هر نود با استفاده از معادله ی Bellman-Ford تخمین های خودش را بهتر می کند و نهایتاً همه به جواب های یکسانی مثل بردار $d_x = [0, 2, 3]$ می رسند که نشان می دهد کم هزینه ترین مسیرها در شبکه تثبیت شده اند.

رفتار الگوریتم در برابر تغییر link cost هم مهم است. وقتی هزینه ی یک لینک کم می شود (مثلاً در مثال $y \rightarrow x$ که از ۴ به ۱ می رسد)، این یک «خبر خوب» است و معمولاً سریع پخش می شود. نود متصل به لینک، تغییر را حس می کند، DV خودش را آپدیت می کند و برای همسایه ها می فرستد؛ آن ها هم که ببینند مسیر ارزان تری پیدا شده، جدولشان را اصلاح می کنند و DV جدید را پخش می کنند. به همین خاطر در اسلایدها جمله ی «good news travels fast» نوشته شده است.

اما وقتی هزینه ی لینک زیاد می شود یا لینک قطع می شود، با «خبر بد» طرفیم و این جا مشکل معروف «count-to-infinity» ظاهر می شود. در مثال چهارنودی با نودهای A, B, C, D ، وقتی A از شبکه جدا می شود، B و C مدت ها فکر می کنند هنوز راهی ارزان به A از طریق هم دیگر وجود دارد. مثلاً B می گوید من از طریق C به A می رسم و C هم می گوید من از طریق B می رسم؛ هر کدام هزینه ی خودش را کمی بالاتر از دیگری می گذارد و این عددها همین طور بالا می روند: ۳، ۴، ۵، ... تا بالاخره به یک infinity از پیش تعیین شده برسند. در مثال دیگر شبکه ی سه نودی x, y, z با افزایش شدید هزینه ی لینک (x, y) به ۶۰، نشان داده می شود که الگوریتم ممکن است ده ها تکرار طول بکشد تا به حالت پایدار برسد؛ علت این است که پیام ها ناهمگام هستند و هر نود فقط distance vector خودش را برای همسایه ها می فرستد، نه کل جدول داخلی اش را، و همین باعث می شود اطلاعات «خراب» مدت زیادی بین نودها پاس شود.

برای کم کردن این مشکلات، تکنیک هایی مثل poison reverse و split horizon معرفی می شوند. در poison reverse، اگر نود Z برای رسیدن به مقصد X از همسایه ی Y استفاده کند، در پیام هایی که به Y می فرستد، فاصله ی خودش تا X را ∞ اعلام می کند. این یعنی به Y می گوید «اگر می خواهی به X برسی، اصلاً روی من حساب نکن»، و به این شکل از ایجاد حلقه ی دو نودی بین Y و Z جلوگیری می شود. split horizon می گوید اگر نودی مسیر یک مقصد را از یک همسایه یاد گرفته، همان مسیر را دوباره برای همان همسایه تبلیغ نکند؛ مثلاً اگر B بداند که مسیر $(E, 2)$ را از A یاد گرفته است، وقتی به A update می فرستد، این مسیر را در پیامش نمی آورد. نسخه ی قوی تر یعنی split horizon with poison reverse این است که این مسیر را با فاصله ی ∞ برمی گرداند تا مطمئن شود همسایه اصلاً روی آن حساب نکند. البته خود اسلایدها تأکید می کنند که این روش ها فقط حلقه های دو نودی را به خوبی کنترل می کنند و برای حلقه های بزرگ تر باید از روش های جدی تری مثل triggered update و مکانیزم های پیچیده تر استفاده کرد. در آخر، یک مقایسه ی کلی بین الگوریتم های link-state (مثل Dijkstra) و الگوریتم های distance-vector ارائه

می‌شود. از نظر پیچیدگی پیام، در LS هر نود اطلاعات link-state خود را در کل شبکه flood می‌کند، و با n نود و E لینک حدود $O(nE)$ پیام رد و بدل می‌شود؛ از نظر محاسباتی هم الگوریتم Dijkstra حدود $O(n^2)$ زمان (یا با پیاده‌سازی بهتر $O(n \log n)$) نیاز دارد. در عوض DV فقط بین همسایه‌ها پیام رد و بدل می‌کند و پیچیدگی پیام به ساختار شبکه و سرعت همگرایی بستگی دارد. از نظر سرعت همگرایی، LS معمولاً قابل پیش‌بینی‌تر است، هرچند ممکن است به‌خاطر وابستگی هزینه‌ها به ترافیک دچار oscillation شود. در DV، زمان همگرایی ممکن است خیلی متفاوت باشد و مشکلاتی مثل حلقه‌های مسیریابی و count-to-infinity وجود دارد. از نظر مقاومت در برابر خطا، اگر یک router در الگوریتم LS هزینه‌ی لینک‌هایش را اشتباه advertise کند، فقط جدول خودش را اشتباه حساب می‌کند، اما بقیه نودها همچنان از تصویر نسبتاً درست شبکه استفاده می‌کنند. در DV، اگر یک نود distance vectorهای غلط بفرستد، چون بقیه نودها مستقیم به آن اعتماد می‌کنند، خطا می‌تواند در کل شبکه پخش شود و برای مدتی زیادی همه را دچار اشتباه کند. به همین خاطر، انتخاب بین LS و DV همیشه یک موازنه بین پیچیدگی، سرعت همگرایی و پایداری در برابر خطا و تغییرات link cost است.

مراجع